SESSIONS

SESSIONS

- PHP session allows you to manage information about a user's session.
- You can write smart applications that can identify and gather information about users.
- Let us say that 50 people are clicking around on the same site, e.g. a web shop, at the same time.
  - Information on what each of them have in their shopping cart would best be stored in **a session**.
  - In order to **identify the individual users**, the server uses a **unique user ID** that is stored in a **cookie**.
- A cookie is a small text file stored on the user's computer.
- Therefore, sessions often require support of cookies in the user's browser.

WHAT IS SESSIONS?

❖A PHP session variable is used to store information about, or change settings for a user session.

❖Session variables hold information about one single user, and are available to all pages in one application.

STARTING A PHP SESSION

❖Before you can store user information in your PHP session, you must first **start up** the session.

❖The code below will register the user's session with the server, allow you to start saving user information, and assign a UID for that user's session.

SYNTAX

```
<?php session_start(); ?>
<html>
<body>
</body>
</html>
```

Note: The session_start() function must appear BEFORE the <html> tag

STORING SESSION VARIABLE

EXAMPLE

```
<?php
      session_start();
   // store session data
   $_SESSION['views']=1;
?>
<html>
<body>
   <?php
      //retrieve session data
      echo "Pageviews=". $_SESSION['views'];
   ?>
</body>
</html>
```

DESTROYING A SESSIONS

- To delete some session data, you can use the `unset()` or the `session_destroy()` function.
- The unset() function is used to free the specified session variable

SYNTAX

```
<?php
      unset($_SESSION['views']);
?>
```

DESTROYING A SESSIONS

By default, session lasts till the user closes the browser, then it dies automatically. But if you want to stop a session,

Completely destroy the session by calling the `session_destroy()` function

SYNTAX

```
<?php
      session_destroy();
?>
```

Note: session_destroy() will reset your session and you will lose all your stored session data.

Login system with sessions: form.php

```
<html>
<head>
<title>Login</title>
</head>
<body>
<form method="post" action="login.php">
<p>Username: <input type="text" name="username" /></p>
<p>Password: <input type="password" name="password"/></p>
<p><input type="submit" value="Let me in" /></p>
</form>
</body>
</html>
```

TUTORIAL LOGIN1

Login system with sessions: login.php

```
<?php
// Check if username and password are correct
if ($_POST["username"] == "php" && $_POST["password"] == "php") {
// If correct, we set the session to YES
 session_start();
 $_SESSION["Login"] = "YES";
 echo "<h1>You are now logged correctly in</h1>";
 echo "<p><a href='document.php'>Link to protected file</a><p/>";
}
else {
// If not correct, we set the session to NO
 session_start();
 $_SESSION["Login"] = "NO";
 echo "<h1>You are NOT logged correctly in </h1>";
 echo "<p><a href='document.php'>Link to protected file</a></p>";
}
?>
```

TUTORIAL LOGIN1

Login system with sessions: document.php

```
<?php
// Start up your PHP Session
session_start();
// If the user is not logged in send him/her to the login form
if ($_SESSION["Login"] != "YES") {
header("Location: form.php");
}
?>

<html>
<head><title>Login</title></head>
<body>
<h1>This document is protected</h1>
<p>You can only see it if you are logged in.</p>
</body>
</html>
```

TUTORIAL LOGIN1

login system with session & database

Create the file below step by step, modified the code then upload to your web server.

config.php

create_user.php

insert_data_user.php

login.php

check_login.php

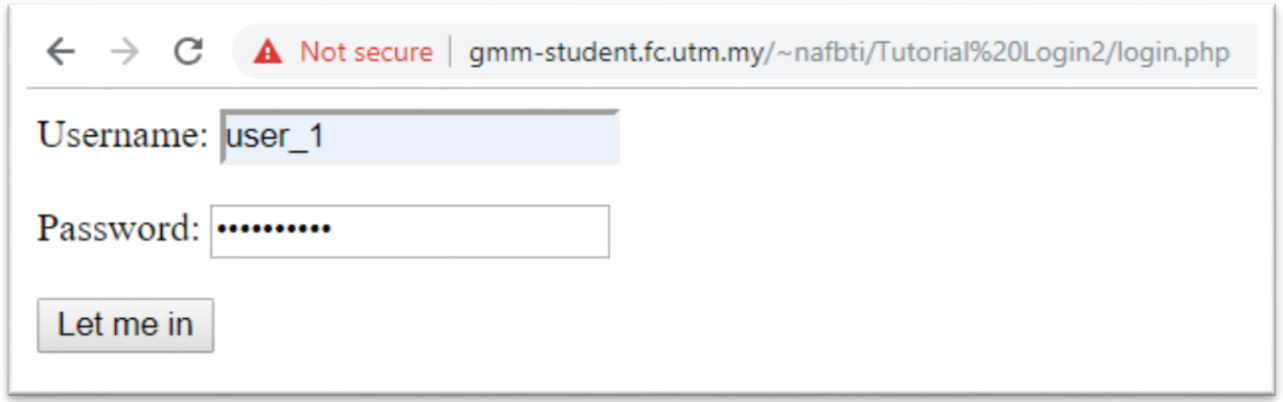
document.php

document_level.php

First things, you need to change the code. Change username, password, db to your web server. Save the file. Next, upload to your web hosting. Lastly, execute the file via your web page.

TUTORIAL LOGIN2

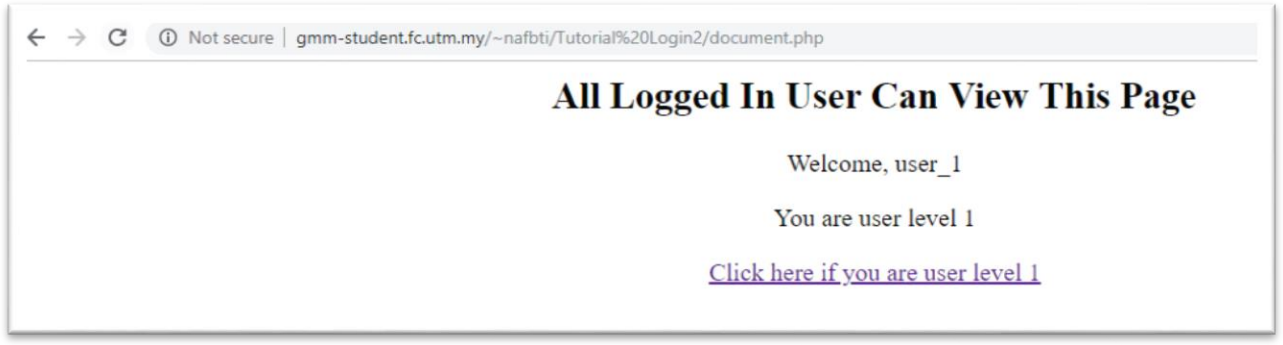
Tutorial Login 2



Try login user_1.



If you user_1, means user level 1 and an authorize user. You can successfully logon. Click the link Proceed to Site.



Tutorial Login 2





If your user level 1, can may proceed to link "Click here if you are user level 1"

COOKIES

WHAT IS COOKIES?

- A cookie is a small text file in which a website can store different information.
- A cookie is often used to **identify a user**.
- Cookies are saved on the user's **hard drive(clients)** and not on the server.
- Most cookies expire (delete themselves) after a predetermined time period, which can range from one minute to several years.
- But the user can also identify and delete any cookies on his/her computer.

WHAT IS COOKIES?

- A cookie is often used to identify a user
- To track certain user details like *(No. Of Visits, names, last visit, etc).*
- The client machine stores such information and sends it to the web server whenever there is a request.
- Cookies data are sent along with the HTTP headers.

HOW TO CREATE A COOKIE?

The `setcookie()` function is used to set a cookie.

SYNTAX

```
setcookie(name, value, expire, path, domain);
```

Note: The `setcookie()` function must appear BEFORE the `<html>` tag.

HOW TO CREATE A COOKIE?

name: The name of your cookie. You will use this name to later retrieve your cookie, so don't forget it!

value: The value that is stored in your cookie. Common values are username(string) and last visit(date).

expiration: The date when the cookie will expire and be deleted. If you do not set this expiration date, then it will be treated as a session cookie and be removed when the browser is restarted.

Expire in 1 hour (because 1 hour has 3600 seconds)

HOW TO CREATE A COOKIE?

EXAMPLE 1

We will create a cookie named `"user"` and assign the value `"Alex Porter"` to it. We also specify that the cookie should expire after one hour:

SYNTAX

```
<?php
        setcookie("user", "Alex Porter", time()+3600);
?>
<html>
.....
```

The expiration time is set to 1 hour (60 sec * 60 min).

HOW TO CREATE A COOKIE?

EXAMPLE 2

Set the expiration time of the cookie in another way. It may be easier than using seconds.

SYNTAX

```
<?php
        $expire=time()+60*60*24*30;
        setcookie("user", "Alex Porter", $expire);
?>
<html>
.....
```

Note: The expiration time is set to a month (*60 sec * 60 min * 24 hours * 30 days*).

HOW TO RETRIEVE A COOKIE VALUE?

The PHP `$_COOKIE` variable is used to retrieve a cookie value.

SYNTAX

```
<?php
    // Print a cookie
    echo $_COOKIE["user"];
    // A way to view all cookies
    print_r($_COOKIE);
?>
```

HOW TO RETRIEVE A COOKIE VALUE?

Retrieve the value of a cookie using global variable $_COOKIE and the isset() function

EXAMPLE 2

Use the `isset()` function to find out if a cookie has been set

SYNTAX

```
<html>
<body>
<?php
    if (isset($_COOKIE["user"]))
      echo "Welcome " . $_COOKIE["user"] . "!<br />";
    else
      echo "Welcome guest!<br />";
?>
</body>
</html>
```

HOW TO RETRIEVE A COOKIE VALUE?

Retrieve the value of a cookie using global variable $_COOKIE and the isset() function

EXAMPLE 3

```
<?php

$cookie_name = "Name";
$cookie_value = "C.Wing";

// Setting the cookie
setcookie($cookie_name, $cookie_value, time()+3600);

// Check if the cookie is set

if(!isset($_COOKIE[$cookie_name])) {
 echo "Cookie named '".$cookie_name."' is not set!";
} else {
 echo "Cookie '" .$cookie_name. "' is set!<br>";
 echo "Value is: " . $_COOKIE[$cookie_name]; //retrieve the cookie value
 }
?>
```

HOW TO DELETE COOKIES

When deleting a cookie you should assure that the expiration date is in the past.

SYNTAX

```
<?php
   // set the expiration date to one hour ago
   setcookie("user", "", time()-3600);
?>
```

Who can read the cookie?

- By default, a cookie can be read at the same second-level domain (e.g. utm.my) as it was created.
- But by using the parameters domain and path, you can put further restrictions on the cookie using the following syntax:

```
setcookie(name, value, expiration time, path, domain);
```

Optional. Specifies the server path of the cookie. If set to "/", the cookie will be available within the entire domain. If set to "/php/", the cookie will only be available within the php directory and all sub-directories of php. The default value is the current directory that the cookie is being set in.

Path

setcookie*(name, value, expiration time, path, domain);*

domain

Optional. Specifies the domain name of the cookie. To make the cookie available on all subdomains of example.com, set domain to "example.com". Setting it to www.example.com will make the cookie only available in the www subdomain.

Example. .

```
<?php

// Setting the cookie: name, value, expiration time, path, domain
setcookie("HTMLTest", "This text is in a cookie!",
time()+86400*30, "/", "");

//The "/" means that the cookie is available in entire website
//for a localhost, domain=""

// Write the information
echo $_COOKIE ["HTMLTest"];

?>
```

Example. .

```
<?php
// Setting the cookie: name, value, expiration time,
//path, domain

setcookie(''HTMLTest'', ''This text is in a cookie!'',
time()+60*60*24, ''/tutorial/php/'', ''utm.my'');

// for a localhost, domain=""
// Write the information to the client

echo $_COOKIE [''HTMLTest''];
?>
```

THE END